



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO



**FACULTAD
DE INGENIERÍA**

Licenciatura en Ciencias de la Computación

Programación Paralela y Distribuida

TEMA: MULTIPLICACIÓN DE MATRICES

Segunda entrega parcial

Laricchia Aida, Nahman Martina y Sorbello Mauro

1. Descripción del problema a resolver

Como se presentó en la primera entrega, el problema consiste en realizar la multiplicación de matrices de grandes dimensiones utilizando inicialmente una versión secuencial y posteriormente desarrollar alternativas paralelas/distribuidas que permitan reducir el tiempo de ejecución.

En esta segunda etapa se desarrollaron dos implementaciones paralelas:

- una versión utilizando únicamente MPI
- y una versión híbrida utilizando MPI + OpenMP.

No se realizaron modificaciones sobre la implementación secuencial presentada anteriormente.

Respecto al análisis realizado en la primera entrega, inicialmente se consideraron distintas estrategias posibles de paralelización, como:

- división por filas y columnas,
- división por bloques,
- y paralelización del bucle interno.

Finalmente, para esta implementación se optó por una estrategia de división por filas utilizando MPI, donde cada proceso recibe un conjunto de filas de la matriz A y realiza la multiplicación contra todas las columnas de la matriz B. Esta decisión se tomó debido a su simplicidad de implementación, buen balance de carga y facilidad para distribuir el trabajo entre procesos.

Además, se decidió explorar una segunda alternativa híbrida utilizando OpenMP junto con MPI, buscando aprovechar paralelismo adicional dentro de cada proceso mediante múltiples hilos de ejecución.

2. Estrategias y decisiones de diseño

2.1 Estrategia implementada con MPI

Para la versión distribuida utilizando MPI se decidió dividir el trabajo por filas de la matriz A.

Segunda entrega Trabajo Final Integrador Laricchia, Nahman y Sorbello

Cada proceso recibe un conjunto de filas de la matriz principal y realiza la multiplicación de dichas filas contra todas las columnas de la matriz B. De esta forma, cada proceso calcula únicamente una parte de la matriz resultado C.

La matriz B se comparte completa entre todos los procesos utilizando `MPI_Bcast`, ya que todos los procesos necesitan acceder a ella para realizar los cálculos correspondientes.

Posteriormente:

- las filas de la matriz A se distribuyen utilizando `MPI_Scatterv`,
- cada proceso calcula su parte de la matriz resultado,
- y finalmente todos los resultados parciales se reúnen utilizando `MPI_Gatherv`.

Se eligió esta estrategia debido a que:

- posee una implementación relativamente sencilla,
- permite un buen balance de carga,
- y aprovecha el paralelismo natural presente en la multiplicación de matrices.

Además, se decidió utilizar `Scatterv` y `Gatherv` en lugar de `Scatter` y `Gather` tradicionales para permitir distribuir correctamente las filas incluso cuando la cantidad de procesos no divide exactamente la dimensión de la matriz.

2.2 Estrategia implementada con MPI + OpenMP

Posteriormente se desarrolló una segunda versión híbrida utilizando MPI junto con OpenMP.

La idea principal fue mantener la distribución de trabajo realizada con MPI entre procesos, pero además paralelizar internamente los bucles de multiplicación utilizando hilos OpenMP.

Para ello se utilizó:

```
#pragma omp parallel for collapse(2)
```

permitiendo paralelizar simultáneamente:

- el recorrido de filas
- el recorrido de columnas

El objetivo de esta propuesta fue analizar si combinar:

- paralelismo distribuido (MPI)
- junto con paralelismo compartido (OpenMP)

permitía mejorar aún más los tiempos de ejecución aprovechando múltiples núcleos dentro de cada proceso.

Sin embargo, en las pruebas realizadas hasta el momento, la versión utilizando únicamente MPI presentó mejores tiempos de ejecución que la implementación híbrida con OpenMP.

Consideramos que esto puede deberse a:

- overhead de creación y sincronización de hilos
- tamaño del problema asignado a cada proceso
- o configuración de recursos utilizada durante las ejecuciones.

3. Pseudocódigo versión secuencial

Algoritmo MultiplicacionMatrices

Entrada:

Matriz A de tamaño $M \times R$
Matriz B de tamaño $R \times N$

Salida:

Matriz C de tamaño $M \times N$

Para i desde 0 hasta M-1 hacer

Para j desde 0 hasta N-1 hacer
C[i][j] = 0

Para k desde 0 hasta R-1 hacer

C[i][j] = C[i][j] + A[i][k] *

B[k][j]

Fin Para

Fin Para

Fin Para

Retornar matriz C

4. Pseudocódigo versión paralela MPI

Inicializar MPI

Obtener rank y cantidad de procesos

Proceso 0:

Generar matrices A y B aleatorias

Compartir matriz B con todos los procesos

Dividir filas de la matriz A entre procesos

Distribuir filas utilizando Scatterv

Cada proceso:

Multiplifica sus filas asignadas
contra todas las columnas de B

Reunir resultados utilizando Gatherv

Proceso 0:

Mostrar tiempo de ejecución

Finalizar MPI

5. Pseudocódigo versión híbrida MPI + OpenMP

Inicializar MPI

Obtener rank y cantidad de procesos

Proceso 0:

Generar matrices A y B aleatorias

Compartir matriz B con todos los procesos

Dividir filas de la matriz A entre procesos

Distribuir filas utilizando Scatterv

Cada proceso:

Paraleliza internamente los bucles
utilizando OpenMP

Multiplifica filas y columnas
simultáneamente mediante hilos

Reunir resultados utilizando Gatherv

Proceso 0:

Mostrar tiempo de ejecución

Finalizar MPI

6. Resultados obtenidos

Se presenta a continuación una captura de pantalla del código secuencial corriendo en el cluster de mulatona y los resultados obtenidos:

Se utilizaron matrices cuadradas de orden 5000 x 5000 con números aleatorios generados a partir de una semilla con valor: 1234 para realizar la multiplicación.

```

cursoppyd15@mulatona:~
GNU nano 5.6.1 salida_289506.txt
=====
JOB MPI
=====
Nodos: 1
Tasks totales: 8
Tiempo paralelo MPI: 188.012 segundos

Job Metrics:
Job ID: 289506
Cluster: bdw
User/Group: cursoppyd15/cursoppyd15
State: COMPLETED (exit code 0)
Nodes: 1
Cores per node: 8
CPU Utilized: 00:25:36
CPU Efficiency: 61.34% of 00:41:44 core-walltime
Job Wall-clock time: 00:05:13
Memory Utilized: 1.24 GB
Memory Efficiency: 4.07% of 30.50 GB (3.81 GB/core)

```

```

cursoppyd15@mulatona:~
GNU nano 5.6.1 salida_mpi289634.txt
=====
JOB MPI
=====
Nodos: 1
Tasks totales: 16
Tiempo paralelo MPI: 95.0877 segundos

Job Metrics:
Job ID: 289634
Cluster: bdw
User/Group: cursoppyd15/cursoppyd15
State: COMPLETED (exit code 0)
Nodes: 1
Cores per node: 16
CPU Utilized: 00:26:00
CPU Efficiency: 95.59% of 00:27:12 core-walltime
Job Wall-clock time: 00:01:42
Memory Utilized: 2.10 GB
Memory Efficiency: 3.44% of 61.00 GB (3.81 GB/core)

```

Segunda entrega Trabajo Final Integrador
Laricchia, Nahman y Sorbello

```

cursoppyd15@mulatona:~
GNU nano 5.6.1 salida_289631.txt
=====
JOB MPI + OPENMP
=====
Nodos: 1
MPI tasks: 4
Threads OpenMP: 2
Tiempo paralelo MPI + OpenMP: 406.818 segundos

Job Metrics:
Job ID: 289631
Cluster: bdw
User/Group: cursoppyd15/cursoppyd15
State: COMPLETED (exit code 0)
Nodes: 1
Cores per node: 8
CPU Utilized: 00:27:10
CPU Efficiency: 49.45% of 00:54:56 core-walltime
Job Wall-clock time: 00:06:52
Memory Utilized: 1.08 GB
Memory Efficiency: 3.54% of 30.50 GB (3.81 GB/core)

```

```

cursoppyd15@mulatona:~
GNU nano 5.6.1 salida_mpi-omp289633.t
=====
JOB MPI + OPENMP
=====
Nodos: 1
MPI tasks: 8
Threads OpenMP: 2
Tiempo paralelo MPI + OpenMP: 220.535 segundos

Job Metrics:
Job ID: 289633
Cluster: bdw
User/Group: cursoppyd15/cursoppyd15
State: COMPLETED (exit code 0)
Nodes: 1
Cores per node: 16
CPU Utilized: 00:30:27
CPU Efficiency: 47.98% of 01:03:28 core-walltime
Job Wall-clock time: 00:03:58
Memory Utilized: 1.25 GB
Memory Efficiency: 2.05% of 61.00 GB (3.81 GB/core)

```